

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

INTRO TO CRITICAL INFRASTRUCTURE FOR IT LEARNERS

LAB 05 — BUILDING A SCADA HUMAN MACHINE INTERFACE

Student Edition

Student name	Sidra Shaikh
Registration No.	C11/26/ACIS/17349
Date	27 September 2026

Authorized ICDFA training environments only
icdfa.edu.ng

Lab 05: Building a SCADA Human Machine Interface

Course: Intro to Critical Infrastructure for IT Learners | **Estimated Duration:** 4–5 hours

Submission deadline: As published in ICDFA E-Campus.

Authorization & Safety: Perform all activities only on the ICDFA-provided virtual machines, packet captures and isolated training networks. Do not bridge these VMs to production networks, scan external systems, or reuse credentials observed in training data.

Learning Outcomes

- Apply basic high-performance HMI and situational-awareness principles.
- Create a tank, level indication, valve and process piping in FUXA.
- Connect the HMI to the OpenPLC tank-fill simulation.
- Bind HMI objects to PLC/Modbus tags and configure simple animation/state indication.
- Demonstrate that the HMI reflects the running process correctly.

Laboratory Environment

System	Expected Address	Account	Password
PLC VM	192.168.1.10	plc	password
SCADA VM	192.168.2.10	scada	password

Note: If the instructor-provided VM image uses updated addressing or credentials, use the values published in E-Campus. Do not alter addressing unless instructed.

Pre-Lab Checklist

- Read the complete lab before beginning.
- Confirm you can access the required VM(s) and that virtualization support is enabled.
- Use the instructor-specified isolated/host-only network.
- Create or restore the clean starting snapshot for this lab.
- Prepare a report document with your name, registration number, course, lab number and date.
- Capture evidence as you progress rather than repeating the lab at the end.

Procedure

Step 1. Start the PLC and SCADA VMs in the isolated laboratory network. Start FUXA on the SCADA VM and open <http://127.0.0.1:1881/editor>.

Step 2. Create a New Project named with your registration number and LAB05, for example 2026XXXX_LAB05.

Step 3. Set a neutral gray background and enable the grid while arranging process graphics.

Step 4. From the Process Engineering objects, place a tank graphic. Add a bar gauge over/inside the tank and set its engineering range to 0–10 ft.

Step 5. Build a simple motor-actuated fill valve using the available valve/shape objects. Add a pipe from the valve to the tank. Keep the normal-state design visually simple.

Step 6. On the PLC VM, load/start the provided TankFillSim.st program in OpenPLC Runtime and confirm the status is Running.

Step 7. In FUXA, create or edit the Modbus TCP connection to the PLC at 192.168.1.10:502.

Step 8. Create/verify the Tank Level tag mapped to the OpenPLC input-register location used by the simulation (%IW80 / Input Register 80 in the teaching image).

Step 9. Create/verify the Tank Valve Open tag mapped to the OpenPLC discrete output used by the simulation (%QX0.0 / Coil 0 in the teaching image).

Step 10. Bind the bar gauge to the Tank Level tag. Configure Min = 0 and Max = 10.

Step 11. Bind the valve-state graphic to Tank Valve Open. Configure the normal/closed representation for value 0 and the open representation for value 1.

Step 12. Use a restrained high-performance style: neutral normal states, clear labels, and attention-drawing color only where it communicates a meaningful abnormal/alarm condition.

Step 13. Enable the PLC/FUXA connection and run the HMI. Observe the tank filling/draining and confirm the valve-state graphic changes with the process.

Step 14. Save/export the FUXA project and capture evidence of both the design view and running HMI.

Required Evidence & Submission

- Screenshot of the HMI in design/edit mode.
- Screenshot of the running HMI with a visible tank-level value.
- Screenshot of the configured PLC connection/tags with sensitive passwords excluded.
- The exported FUXA project file.
- A short design rationale explaining how the interface supports situational awareness.

Recommended filename: <RegistrationNo>_LAB05_Building_a_SCADA_Human_Machine_Interface.pdf plus the required project/capture file(s).

Knowledge Check

1. What is a P&ID and how does it influence HMI layout?

Response: _____

2. Why do high-performance HMIs avoid unnecessary animation and excessive color?

Response: _____

3. How does the Tank Level tag differ from the Tank Valve Open tag in data type and behavior?

Response: _____

4. What could happen operationally if an HMI displays stale or incorrect PLC data?

Response: _____

Marking Rubric — 100 Marks

Criterion	Marks	Standard
Technical execution	50	Lab procedure completed correctly; required configuration/analysis works as intended.
Evidence	20	Screenshots/files are complete, readable, relevant and clearly labelled.
Analysis & knowledge check	20	Responses demonstrate understanding of the industrial-control concepts observed.
Professional reporting	10	Clear structure, correct terminology, concise writing, file naming and submission compliance.

Submission Integrity: Evidence must come from the student's own assigned laboratory environment. Screenshots should show enough context to verify the activity. Do not submit another learner's screenshots or project files.

Solution Below:



1. Objective

This lab applies high-performance HMI and situational-awareness principles by building a tank-fill process graphic in FUXA, connecting it to the OpenPLC tank-fill simulation over Modbus TCP, and binding HMI objects (a tank level bar gauge and a fill-valve indicator) to live PLC/Modbus tags.

2. Procedure Summary

A new FUXA project was created with a neutral gray background and grid enabled. A tank, bar gauge, motor-actuated fill valve (valve base, rectangle and connecting line) and process piping were placed using the Process Engineering and Controls graphic objects. On the PLC VM, since no pre-made TankFillSim.st file was available, a Structured Text tank-fill simulation program was written directly and compiled in OpenPLC Runtime, correctly exposing TankLevel_LIT at Input Register 80 (%IW80) and TankValveOpen at Coil 0 (%QX0.0) as located, Modbus-addressable variables. A Modbus TCP connection to the PLC was configured in FUXA, both tags were created against this connection, the bar gauge was bound to TankLevel_LIT (Min 0, Max 100, matching the simulation's reporting range), and the valve graphic was animated using a duplicated, colour-changed overlay shape that is hidden or shown depending on the TankValveOpen value.

2.1 Issues encountered and resolved

- No ready-made TankFillSim.st was available on the PLC VM for this lab, consistent with the same gap found in Lab 04. The Structured Text program was therefore authored directly, declaring TankValveOpen, TankLevel_LIT and the other simulation variables as VAR_GLOBAL nested inside the CONFIGURATION block (rather than as ordinary program-local variables), which is the placement IEC 61131-3 and OpenPLC's matiec compiler require for a variable to be exposed as a located, Modbus-addressable point. The program compiled with no errors, and the compiler's own glueVars output confirmed __IW80 (INT) and __QX0_0 (BOOL) were generated as located variables, later verified live via OpenPLC's Monitoring page.
- FUXA's Connections dialog initially offered only WebAPI, MQTTclient and internal device types, with no Modbus option. Investigation via FUXA's Plugins page (127.0.0.1:1881/plugins) showed Modbus TCP is shipped as an installable plugin (modbus-serial) on this training image rather than a built-in driver, and it was not yet installed.
- Installing the plugin required outbound internet access, which the isolated SCADA VM did not have on its primary Internal Network adapter. A second, temporary NAT adapter was added to the VM to reach the public npm registry, the modbus-serial package was installed manually (npm install modbus-serial@8.0.19), and FUXA was restarted, after which Modbus TCP appeared as a connection type.
- After configuring the Modbus connection, the PLC remained unreachable (TCP connection refused). Diagnosis showed the SCADA VM's Internal Network interface had no route to the PLC VM's documented address: the lab's addressing table places the PLC at 192.168.1.10 and the SCADA VM at 192.168.2.10, two different subnets with no router between them on an isolated Internal Network. Assigning the SCADA VM's interface an address on the PLC's own subnet (192.168.1.20/24) resolved connectivity, confirming both VMs were correctly wired to the same virtual network and the issue was addressing rather than topology.
- Once connected, PLC tag values were confirmed live and matching between OpenPLC's own Monitoring page and FUXA's tag table, but the tank graphic still did not animate. The bar gauge's tag binding had been mistakenly applied to a different, mis-selected shape (the valve base) rather than the actual bar gauge object; re-binding the correct shape resolved the animation.

These issues were diagnosed and resolved directly rather than blocking the lab; they are documented here as they reflect genuine environment/network troubleshooting consistent with real SCADA commissioning work.

3. Required Evidence

3.1 HMI in design/edit mode

The FUXA editor showing the completed process graphic: tank, bar gauge, fill valve (base, actuator and connecting line) and piping, built on a neutral gray background.

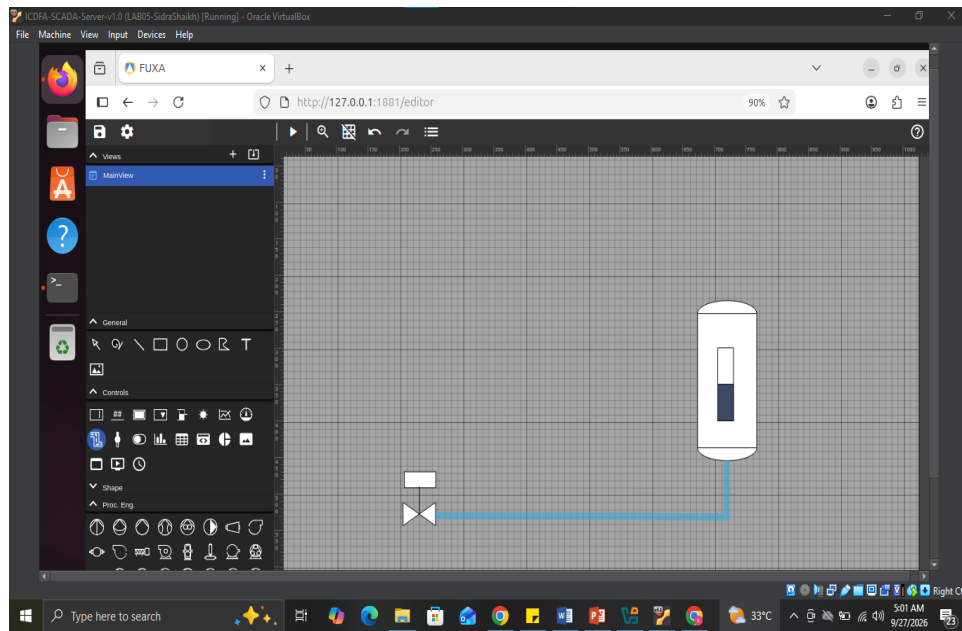


Figure 1. FUXA editor — HMI design/edit view.

3.2 Running HMI with a visible tank-level value

The HMI in Play/runtime mode, with the bar gauge reflecting a live, mid-range tank level read from the PLC over Modbus TCP, and the fill valve graphic showing its live open/closed state.

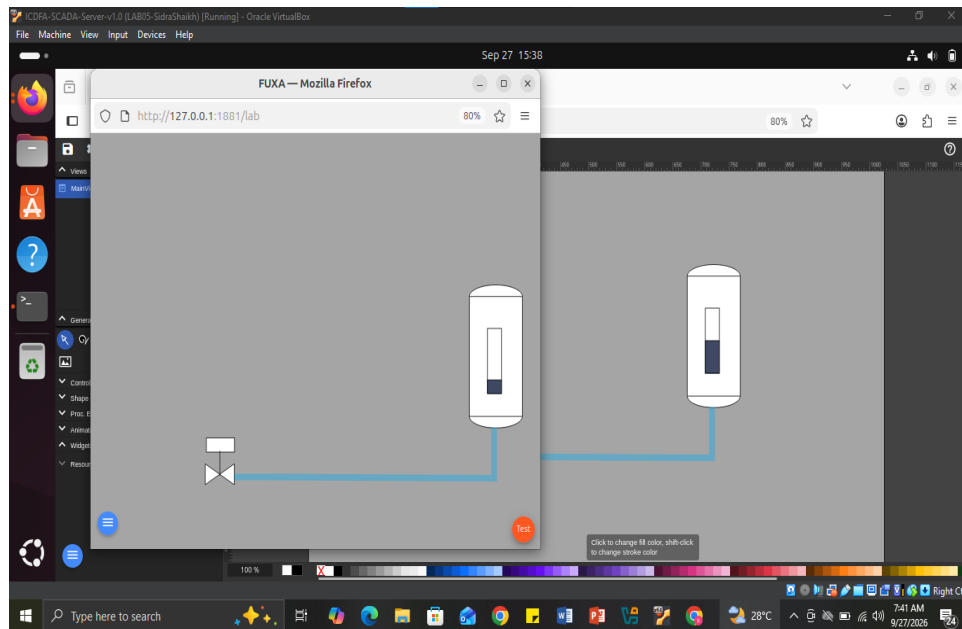


Figure 2. Running HMI showing a live tank-level value.

3.3 Configured PLC connection and tags

The OpenPLC connection configured as Modbus TCP to 192.168.1.10:502 (no credentials required or displayed, consistent with the Modbus protocol), and the resulting tag table confirming TankValveOpen (Coil 0) and TankLevel_LIT (Input Register 80) updating live, matching OpenPLC Runtime's own Monitoring page side by side.

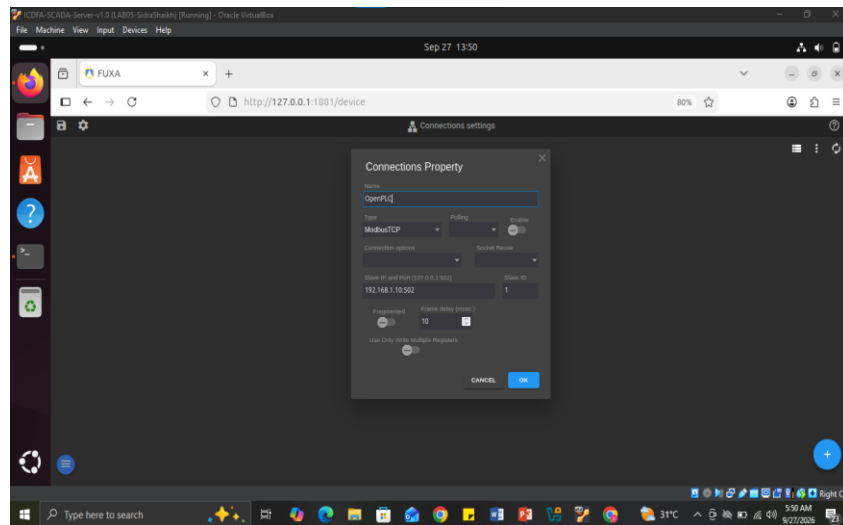


Figure 3. FUXA Modbus TCP connection configuration (no credentials shown).

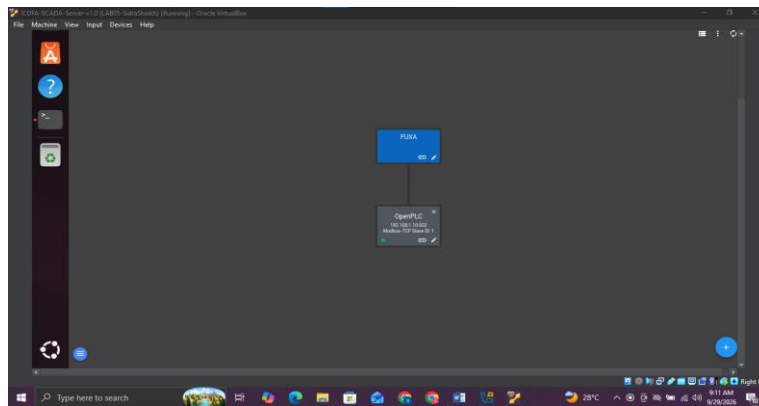


Figure 3b. OpenPLC connection tile showing a live, connected state (green status indicator).

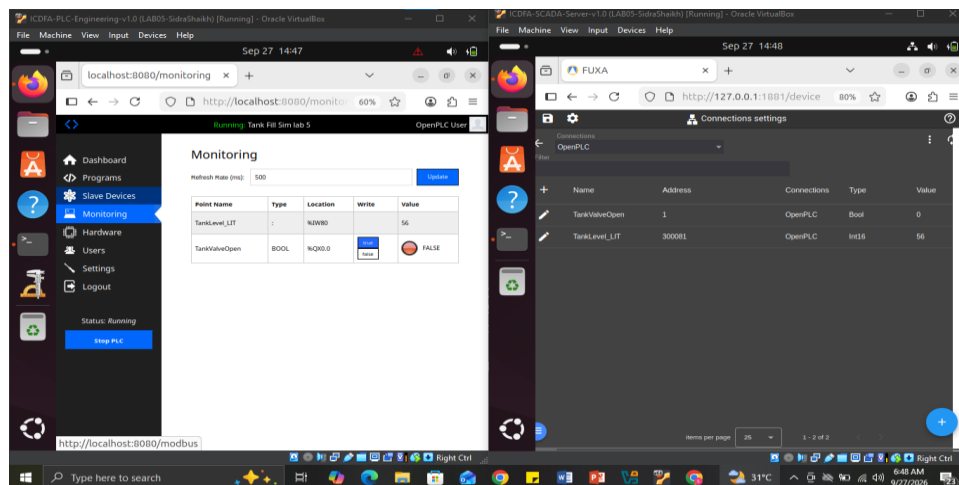


Figure 4. FUXA tag table (left: OpenPLC Monitoring; right: FUXA tags) confirming live, matching values.

3.4 Tank level animation sequence

Three consecutive frames of the running HMI, one minute apart, showing the bar gauge falling as the simulated tank drains, confirming the gauge tracks the live PLC value continuously rather than showing a single static snapshot.

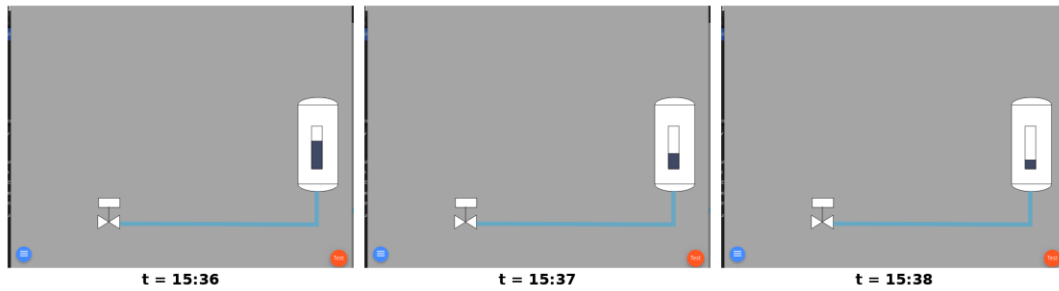


Figure 5. Tank level bar gauge falling across three consecutive live frames (15:36-15:38).

3.5 PLC VM running the compiled program

OpenPLC Runtime's own Dashboard on the PLC VM, confirming Status: Running with the compiled Tank Fill Sim lab 5 program loaded, and the Runtime Logs showing the Modbus server issued and listening on port 502, the data source FUXA is polling.

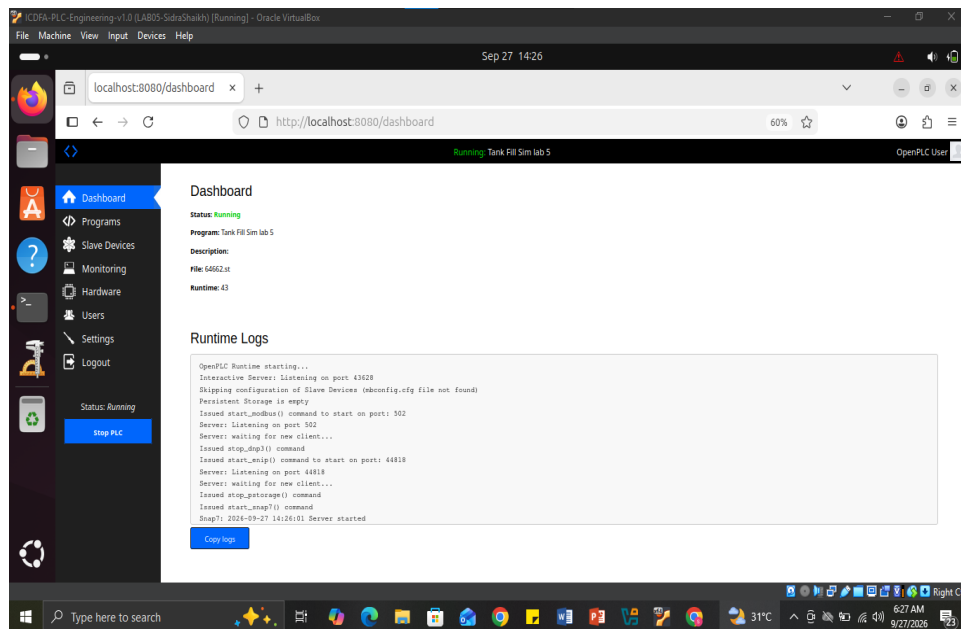


Figure 6. OpenPLC Runtime dashboard on the PLC VM, running the compiled Structured Text program.

3.6 Structured Text program compiled without errors

The self-authored TankFillSim.st, with TankValveOpen, TankLevel_LIT and the simulation's other variables declared as VAR_GLOBAL inside CONFIGURATION, compiling cleanly. The compiler's glueVars output (varName: __IW80 varType: INT and varName: __QX0_0 varType: BOOL) confirms both points were generated as located, Modbus-addressable variables, not just ordinary program variables.

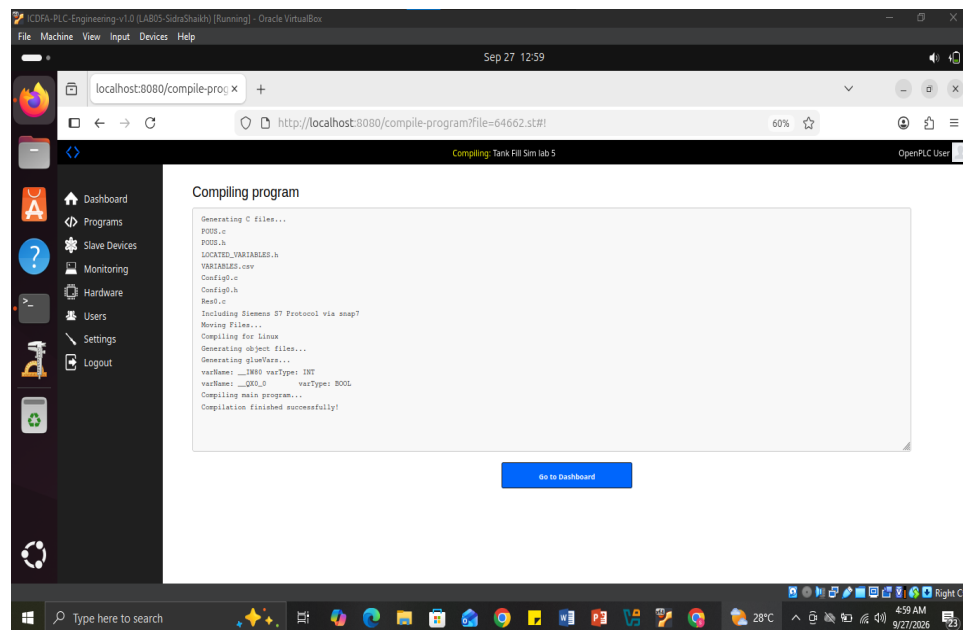


Figure 7. OpenPLC Runtime compile log confirming TankFillSim.st compiled successfully with located variables exposed.

4. Design Rationale-Situational Awareness

The HMI follows high-performance HMI conventions: a neutral gray background with simple, mostly monochrome graphics keeps the screen visually quiet under normal operation, so that the operator's attention is not drawn by decorative motion or unnecessary colour. The tank and valve are represented in a P&ID-style abstraction (tank outline, bar gauge, valve symbol and piping) rather than a literal or animated illustration, keeping the display legible during long shifts.

Situational awareness is supported specifically through the bar gauge's engineering range (0-100, matching the simulation's actual reporting scale) rather than an unscaled raw number, so the operator can judge tank level in context rather than reading an ambiguous value. The valve indicator uses a deliberate colour change (white to dark gray) tied directly to the real discrete PLC state (TankValveOpen), rather than a generic animation, so that the one colour change present on the screen carries real operational meaning: the valve's actual open/closed condition. This selective, restrained use of colour and motion is intended to draw the operator's eye only when the process state itself changes, consistent with high-performance HMI design principles.

5. Knowledge Check

1. What is a P&ID and how does it influence HMI layout?

A Process and Instrumentation Diagram (P&ID) is the engineering drawing used to design and build an industrial process, showing its instrumentation (sensors, valves, piping) and the indications associated with it. HMI layout traditionally follows the P&ID's graphical representation, so the screen mirrors the real process arrangement, for example showing the fill valve, piping and tank level indication in the same relative positions and relationships an engineer would see on the P&ID, making the HMI intuitive to operators already familiar with the process drawing.

2. Why do high-performance HMIs avoid unnecessary animation and excessive color?

Operators often work long shifts monitoring these screens continuously, so a busy, brightly coloured or constantly animated display causes fatigue and desensitises the operator to visual cues over time. High-performance HMI design keeps normal-state screens deliberately plain (dull backgrounds, gray-scale contrast) so that when something does draw the eye, such as a colour change or alarm indication, it reliably means the process needs attention, rather than competing with decorative movement that carries no operational meaning.

3. How does the Tank Level tag differ from the Tank Valve Open tag in data type and behavior?

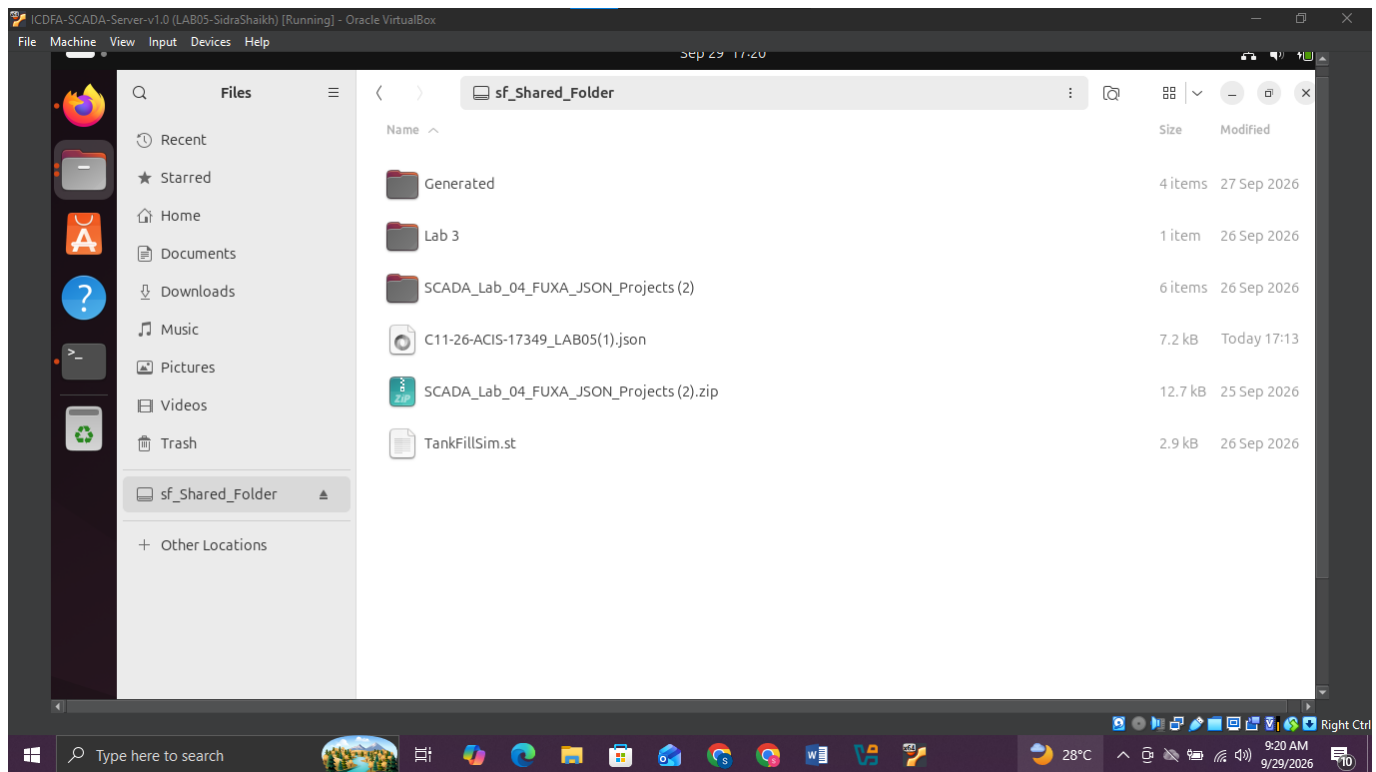
Tank Level (TankLevel_LIT) is an analog/integer value read from an Input Register, representing a continuously varying measurement (the simulated tank level, reported roughly in the 10-100 range) that changes gradually over time as the tank fills or drains. Tank Valve Open (TankValveOpen) is a discrete Boolean value read from a Coil, representing a simple two-state condition, open or closed, that changes abruptly rather than gradually. This is why the two tags are bound differently in the HMI: the level tag drives a continuous bar gauge, while the valve tag drives a discrete show/hide state change.

4. What could happen operationally if an HMI displays stale or incorrect PLC data?

An operator relying on stale or incorrect data could make decisions, or fail to intervene, based on a process state that no longer reflects reality, for example believing a tank is at a safe level when it is actually overfilling, or believing a valve is closed when it is in fact open. This could lead to equipment damage, spills, unsafe conditions, or missed alarms, since the operator's entire awareness of the physical process depends on the HMI accurately reflecting live PLC data; this is exactly why confirming that FUXA's polled values matched OpenPLC's own live values (Section 3.3) was treated as a required verification step in this lab, not just a cosmetic check.

6. Submission

Export the FUXA project (Save icon > Export/Download project) and submit it alongside this report.



Below is the github link for the FUXA file:

https://github.com/SidraShaikh-2/ICDFA-Assignments/tree/main/ACICP301%E2%80%9494%20ICS%20%26%20SCADA%20Systems/C11-26-ACIS-17349_LAB05_TankFillHMI_ProjectFiles